Pérez Aguiar Oropeza Gabriel Emiliano
UNAM / IIMAS / LCD
Semestre 2026-2
Reconocimiento de Patrones

Práctica 2: Extracción de características

1 Objetivos

Implementar el descriptor **Patrones Binarios Locales** en código para extraer características de texturas y clasificarlas utilizando un modelo k-NN. El ejercicio cubre:

- Cálculo del mapa patrones binarios locales.
- Extracción de histogramas de características.
- Clasificación y evaluación de resultados.

2 Introducción

2.1 Histogramas

Los histogramas constituyen la base de varias técnicas de procesamiento en el dominio espacial. La manipulación de los histogramas es usada de manera eficiente en el realce o mejoramiento de la calidad de una imagen. La información estadística obtenida a partir de los histogramas se utiliza en diversas aplicaciones como la compresión y segmentación de imágenes. La facilidad con la que se pueden calcular los histogramas usando software y su bajo consumo de recursos de hardware, han hecho de esta herramienta una de las más usadas en el procesamiento en tiempo real.

El histograma de una imagen es la representación gráfica de la distribución de datos que existe de las distintas tonalidades de grises con relación al número de pixeles o porcentaje de los mismos, es decir, un histograma representa la frecuencia relativa de ocurrencia de los niveles de gris. La representación ideal de un histograma ideal sería la de una recta horizontal, ya que eso indica que todos los posibles valores de grises están distribuidos de manera uniforme.

La ecualización del histograma es una técnica que sirve para obtener un histograma uniforme de tal manera que los niveles de gris son distribuidos sobre la escala y un número igual de pixeles son colocados en cada nivel de gris. Para un observador, esta ecualización hace que las imágenes se vean más balanceadas y con mejor contraste. Como consecuencia, una imagen ecualizada permite que ciertos detalles sean visibles en regiones oscuras o brillantes.

2.2 Vecindades de pixeles

Un pixel p en las coordenadas (x, y) tiene 4 vecinos horizontales y 4 verticales, cuyas coordenadas están dadas por:

$$(x+1, y), (x-1, y), (x, y+1), (x, y-1)$$

Este grupo de pixeles se anota con $N_4(p)$. Así mismo, las vecindades diagonales con el punto (x, y) se anotan como $N_D(p)$, y sus coordenadas son:

$$(x+1, y+1), (x+1, y-1), (x-1, y+1), (x-1, y-1)$$

Para definir de forma adecuada el concepto de vecindad, es necesario revisar el de adyacencia. Dos pixeles son adyacentes si, y sólo si, tienen en común una de sus fronteras, o al menos una de sus esquinas. La figura 1 muestra pixeles adyacentes.

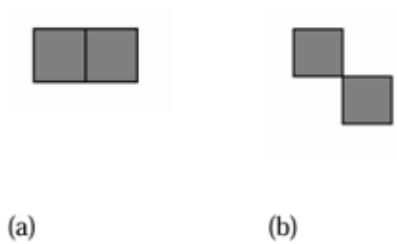


Figura 1: Pixeles adyacentes, (a) adyacentes por frontera, (b) adyacentes por esquina.

Dos pixeles son vecinos si cumplen con la definición de adyacencia. Si los pixeles comparten una de sus fronteras, se dice que los mismos son vecinos directos; si solo se tocan en una de sus esquinas, se llaman vecinos indirectos.

Una vecindad de un pixel p_0 denotado como V_p, es una submatriz M_{KL} de tamaño KxL, con K y L enteros impares pequeños, contenida en la matriz imagen (j_{MN}), la cual está formada por un número finito de pixeles vecinos o no de p_0.

$$V_p = \{p : p \in M_{KL}\}; M_{KL} \subset i_{MN}; K = L = 3, 5, 9$$

Para ilustrar las definiciones anteriores, se puede observar en la figura 2, vecindades de 4 y de vecindades 8, la primera formada por pixeles que son vecinos directos, mientras que la vecindad de 8 está formada tanto por vecinos directos como indirectos.

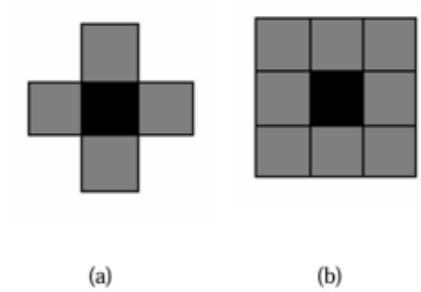


Figura 2: Vecindades, (a) vecindades de 4, (b) vecindades de 8.

En el desarrollo de las técnicas de procesamiento de imágenes que involucren el análisis de una determinada región de la escena digital, es posible encontrar vecindades de $5x5$ y hasta $9x9$; básicamente la definición de las dimensiones de la matriz vecindad depende de la técnica que se esté desarrollando.

2.3 Patrones Binarios Locales (LBP)

El algortimo de patrones binarios locales (LBP por sus siglas en inglés: Local Binary Pattern) es un algoritmo muy usado en visión por computadora para describir la textura de una imagen. Funciona comparando la intensidad de un pixel con la de sus vecinos y generando un código binario que representa el patrón local. El algoritmo de LBP calcula una puntuación para cada pixel, comparándolo con sus vecinos. Matemáticamente, para un pixel central g_c y P vecinos en un radio R:

$$LBP_{P,R} = \sum_{p=0}^{P-1} s(g_p - g_c)2^p$$

Donde $s(x)$ es la función signo:

- $s(x) = 1$ si $x \geq 0$
- $s(x) = 0$ si $x < 0$

Para entender la técnica de LBP, debemos mirar dentro de una ventana de $3x3$ pixeles. El algoritmo convierte la textura local en un número entero simple.

Mecánica de Cálculo Manual (Invariante a la Rotación):

1. **Umbralización:** Compara cada uno de los 8 vecinos ($P = 8, R = 1$) con el pixel central. Si el vecino es mayor o igual, se asigna '1'; de lo contrario, se asigna '0'.
2. **Ponderación y suma:** Los bits resultantes se multiplican por potencias de 2 ($2^0, 2^1, \ldots, 2^7$) siguiendo un orden (por ejemplo, sentido horario comenzando arriba a la izquierda) y se suman para obtener el valor LBP decimal.

3 Desarrollo

3.1 Extracción de Características con LBP para Clasificación de Texturas

1. **Descarga y organiza el conjunto de imágenes.**

   Se utilizó el conjunto de datos Flickr Material Database (FMD), compuesto por 1,000 imágenes de texturas distribuídas equitativamente en 10 clases: carpet, foliage, fabric, leather, metal, paper, plastic, stone, water y wood.

   Posteriormente, para adquirir los datos desde el sitio de Kaggle se utilizó la biblioteca 'kagglehub', y se emplearon las bibliotecas 'os' y 'glob' para organizar las imágenes por categoría en un diccionario.

   La información se estructuró en una lista `categories`, que almacena los nombres de las clases, y un diccionario `map`, que vincula cada categoría con sus respectivas rutas de imágenes.

2. **Carga las imágenes y conviertelas a escala de grises.**

   Se empleó la biblioteca 'OpenCV' para cargar las imágenes en formato RGB. Estas se almacenaron en un diccionario denominado `images`, donde cada clave representa una categoría y su valor es la lista de imágenes correspondientes.

Después, se generó el diccionario `gray_images` para almacenar las imágenes convertidas a escala de grises. La transformación se realizó iterando sobre el diccionario `images` y aplicando la función `cv2.cvtColor()` de OpenCV con el parámetro `cv2.COLOR_RGB2GRAY`.

Posteriormente, por medio de la biblioteca `matplotlib.pyplot` se visualizó la primera imagen de la clase 'wood' para verificar la correcta carga y conversión del conjunto de datos.

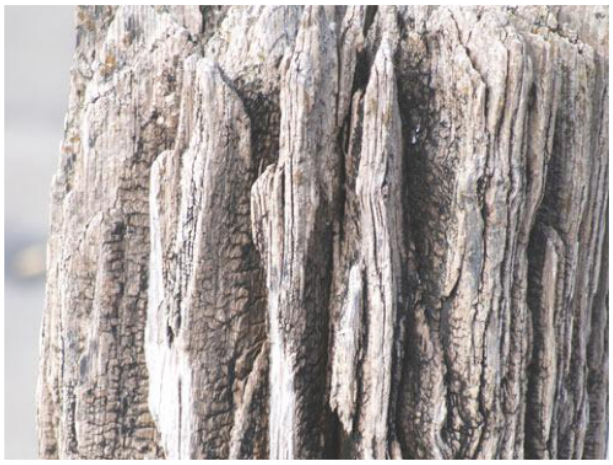


Fig. 1. Imagen de la clase 'wood'.

Fig. 2. Imagen de la clase 'wood' en escala de grises.

Finalmente, se visualizó una imagen en escala de grises por cada clase junto con su histograma correspondiente. Esto permitió observar la distribución de intensidades de píxeles en las distintas categorías. Algunos de estos resultados se muestran a continuación.

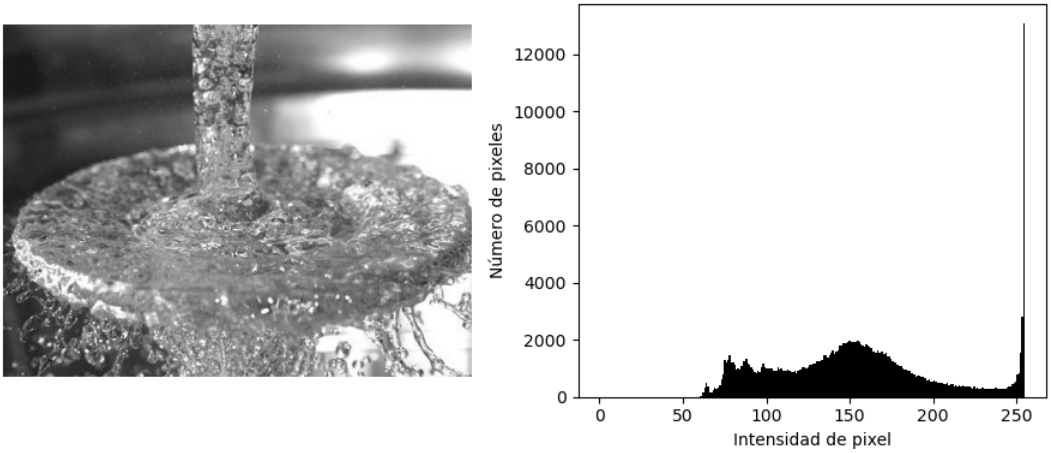


Fig. 3. Imagen de la clase 'water' junto con su histograma correspondiente.

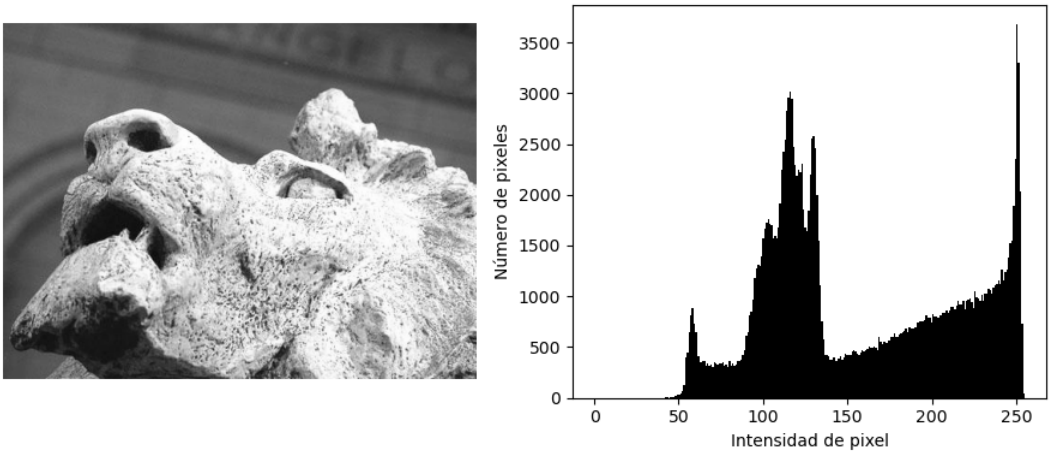


Fig. 4. Imagen de la clase 'stone' junto con su histograma correspondiente.

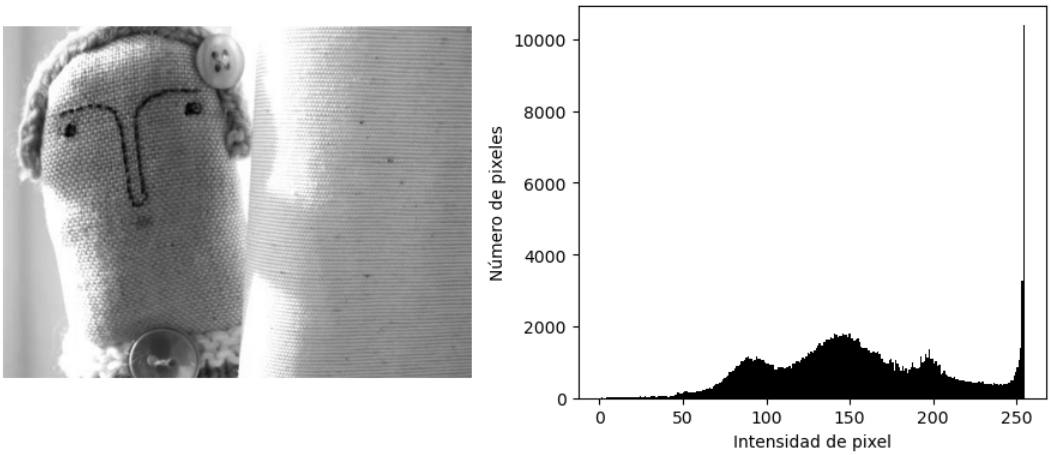


Fig. 5. Imagen de la clase 'fabric' junto con su histograma correspondiente.

3. **Implementación del algoritmo LBP.**

   Se implementaron dos funciones para calcular el valor LBP de un pixel en una imagen. La primera función, `binary_pattern`, simplemente compara el valor de intensidad del pixel central con un pixel vecino, y devuelve 1 si el vecino es mayor o igual, o 0 en el caso contrario. La segunda función, `lbp_value`, compara el patrón binario del pixel central con sus 8 vecinos. Los unos y ceros resultantes son agrupados como un número binario. Este número, el cual representa el valor lbp del pixel central, es convertido y devuelto en su su forma decimal.

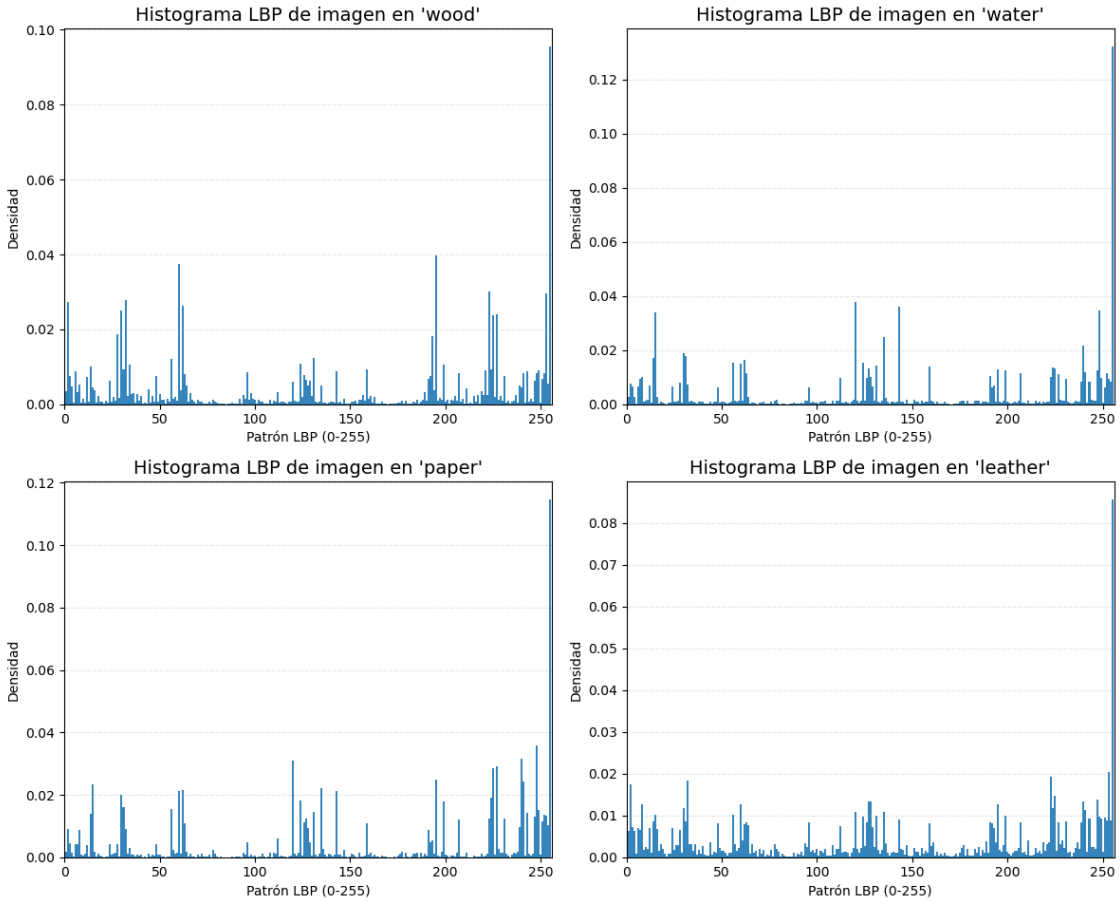
   Finalmente, la función principal `lbp`, crea un arreglo del mismo tamaño de la imagen a procesar. Después, itera sobre todos los pixeles (i,j) de la imagen que no estén en el borde y les aplica la función `lbp_value`. El resultado de esta función es guardado en el espacio (i,j) del arreglo, y la función devuelve el arreglo o imagen LBP.

4. **Convertir cada imagen LBP en un histograma de 256 bins (para valores 0-255).**

   Se iteró sobre las imágenes en `gray_images`, y para cada una de estas, se aplicó el algoritmo del inciso anterior. Las imágenes LBP resultantes se almacenaron en el diccionario `lbp_images`.

   Para construir el histograma de 256 bins, se utilizó la función `cv2.calcHist()` para contar cuántas veces aparece cada valor en una imagen LBP. El histograma se convirtió a un vector unidimensional con `.flatten()` y se normalizó dividiendo este entre su suma. Finalmente, el histograma normalizado se agrega a un diccionario `lbp_hists`, esto para todas las imágenes LBP.

   A continuación se muestran los histogramas LBP de una imagen por cada categoría.



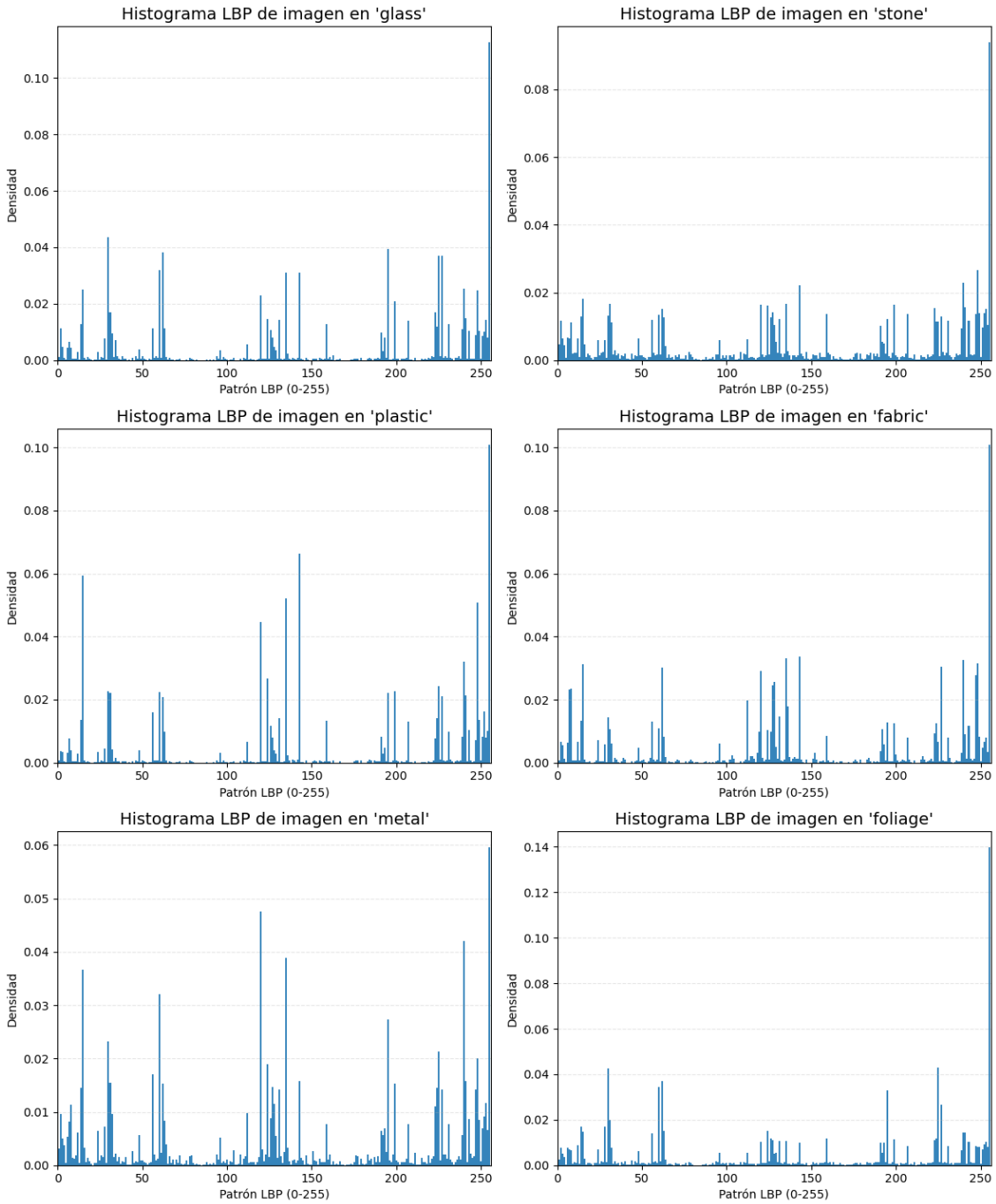


Fig. 6. Histogramas LBP de cada clase.

Podemos observar patrones en los histogramas entre distintas categorías:

- Los histogramas en 'paper', 'water', 'glass' y 'foliage' presentan picos más altos en el valor 255 y muy poco ruido en valores intermedios.
- Los histogramas en 'metal', 'fabric' y 'plastic' muestran picos muy marcados en el centro.
- Los histogramas en 'stone', 'leather' y 'wood' presentan una distribución más uniforme que las otras categorías, con muchos picos pequeños distribuídos por todo el eje X.

5. **Clasificación y Evalucación.**

(a) Se dividieron los datos de los histogramas LBP en entrenamiento (70%) y prueba (30%).

Antes de realizar esta tarea, se separaron los datos en un arreglo de histogramas X y un arreglo de etiquetas y. Después, se utilizó la función `train_test_split` de scikit-learn para separar los datos. Se transformaron las etiquetas en números para que los modelos puedan procesarlas. Finalmente, se normalizaron los datos con el módulo `StandardScaler` de sklearn.

(b) Se entrenaron modelos para clasificar las imágenes basándose en sus histogramas LBP. Después, se evaluó la exactitud de la clasificación.

Utilizando los datos escalados (`X_train_std`) y las etiquetas numeradas (`y_train_encoded`), se entrenaron modelos de clasificación KNN y SVM (Máquinas de Soporte Vectorial).

(c) Se calculó la exactitud de clasificación de ambos modelos en los datos de prueba.

Mediante la función `classification_report`, se imprimió un reporte de clasificación para ambos modelos, mostrando estadísticas como la precisión o el f1-score.

Resultados de KNN:

	precision	recall	f1-score	support
fabric	0.17	0.22	0.19	23
foliage	0.97	0.80	0.88	35
glass	0.22	0.58	0.31	24
leather	0.30	0.24	0.26	34
metal	0.19	0.17	0.18	29
paper	0.40	0.16	0.23	37
plastic	0.43	0.33	0.38	30
stone	0.43	0.39	0.41	33
water	0.26	0.50	0.34	20
wood	0.72	0.37	0.49	35
accuracy			0.37	300
macro avg	0.41	0.38	0.37	300
weighted avg	0.44	0.37	0.38	300

Fig. 7. Evaluación de la clasificación con KNN.

Resultados de SVM:

	precision	recall	f1-score	support
fabric	0.25	0.35	0.29	23
foliage	0.97	0.89	0.93	35
glass	0.31	0.54	0.39	24
leather	0.38	0.32	0.35	34
metal	0.23	0.21	0.22	29
paper	0.40	0.22	0.28	37
plastic	0.35	0.30	0.32	30
stone	0.51	0.61	0.56	33
water	0.48	0.55	0.51	20
wood	0.65	0.57	0.61	35
accuracy			0.46	300
macro avg	0.45	0.45	0.45	300
weighted avg	0.47	0.46	0.46	300

Fig. 8. Evaluación de la clasificación con SVM.

El modelo SVM realizó un mejor trabajo clasificando los datos de prueba. Sin embargo, mientras que clases como 'wood' lograron una clasificación casi perfecta, otras categorías como 'fabric' o 'metal' obtuvieron un mal rendimiento.

(d) Se visualizaron imágenes LBP para entender qué patrones se están detectando.

Para entender por qué clases como 'foliage' y 'stone' tuvieron buena exactitud de clasificación, se visualizaron 3 histogramas LBP de ambas clases y se compararon sus patrones.

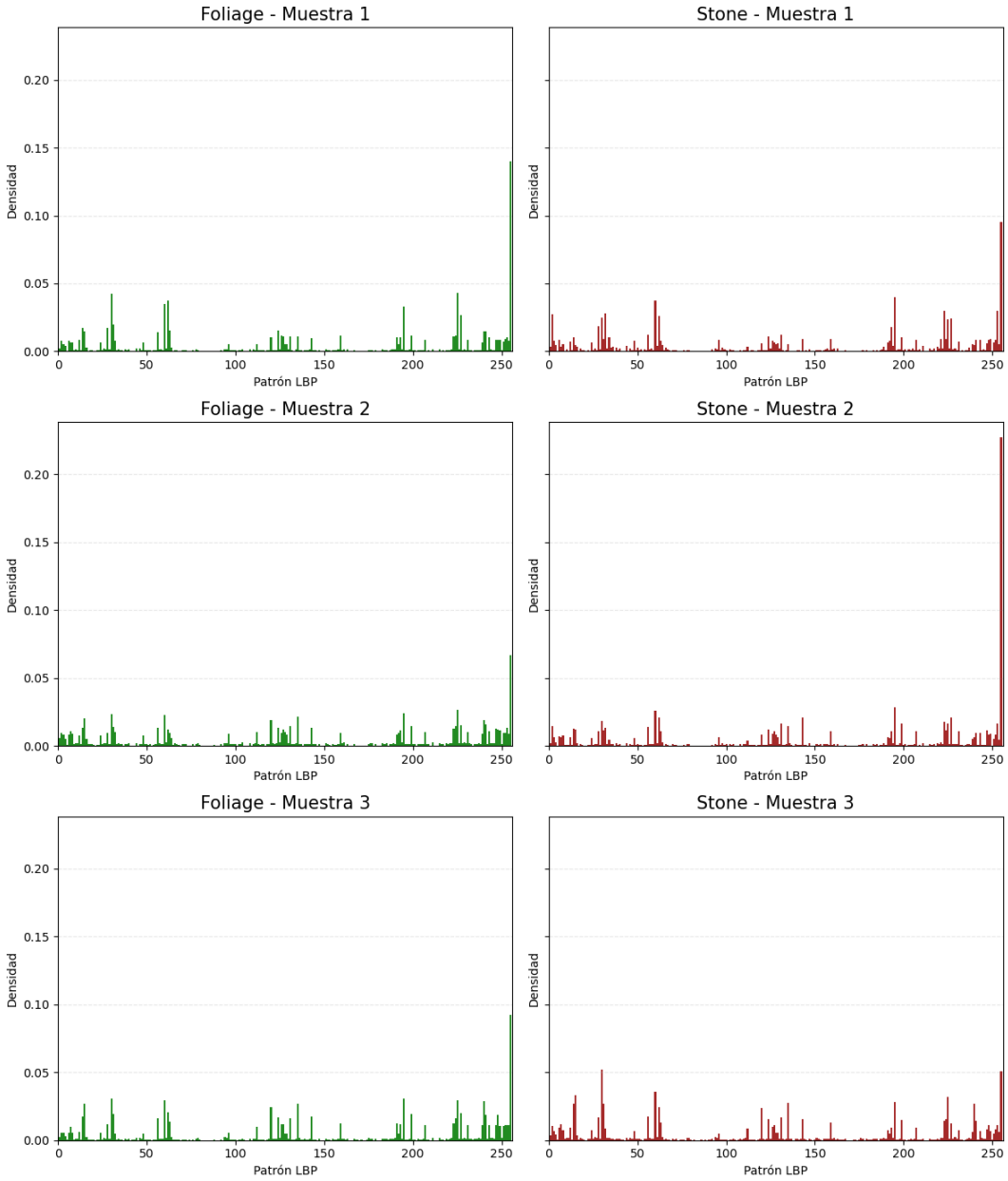


Fig. 9. Imágenes LBP de las clases 'foliage' y 'stone'.

Se puede observar que los histogramas de 'foliage' presentan una distribución de los picos frecuente en todo el rango de valores, mientras que en los histogramas de 'stone', hay más espacios vacíos entre grupos de picos. Esto muestra diferencias concretas entre ambos patrones.

(e) Se mostró la matriz de confusión de las clases predichas contra las clases verdaderas.

Finalmente, se visualizó la matriz de confusión de las clases predichas contra las clases verdaderas utilizando la función `confusion_matrix` de sk-learn. La matriz utiliza las predicciones del modelo SVM, pues fue el que logró un mejor rendimiento. Los resultados se muestran a continuación:

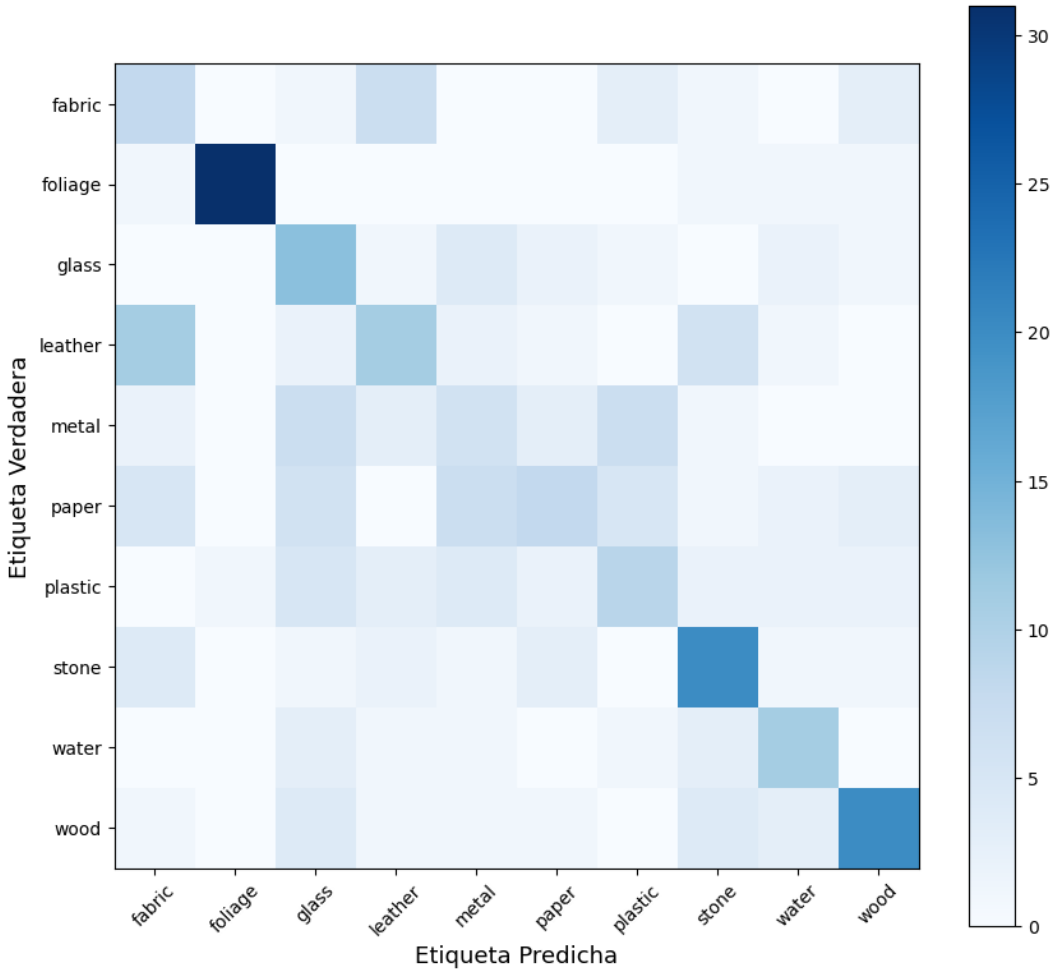


Fig. 10. Matriz de confusión para la clasificación de imágenes utilizando SVM.

6. **Clasificación con el algoritmo LBP para distintos parámetros.**

Se realizó el proceso de conversión de imágenes a arreglos LBP, utilizando diferentes configuraciones de parámetros: $[P = 16, R = 2]$ y $[P = 24, R = 3]$, donde P representa el número de pixeles vecinos y R el radio de la vecindad.

Para ello, se empleó la función `local_binary_pattern` de la biblioteca scikit-learn. Se seleccionó el método 'uniform' para optimizar la eficiencia computacional, pues de lo contrario, el algoritmo generaría histogramas de $2^16 = 65536$ y $2^24 = 16777216$ bins. Mediante el uso de este parámetro, estos contenedores se reducen a $P + 2$. Posteriormente, se graficaron los histogramas para ambas configuraciones y por cada categoría.

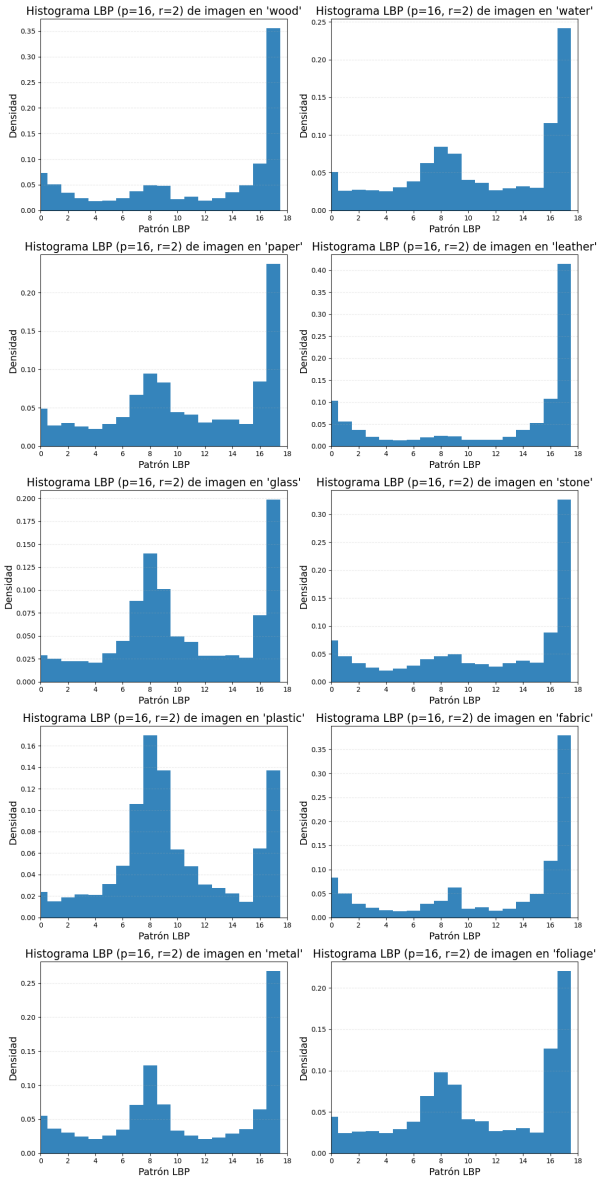


Fig. 11. Histogramas LBP $(p = 16, r = 2)$

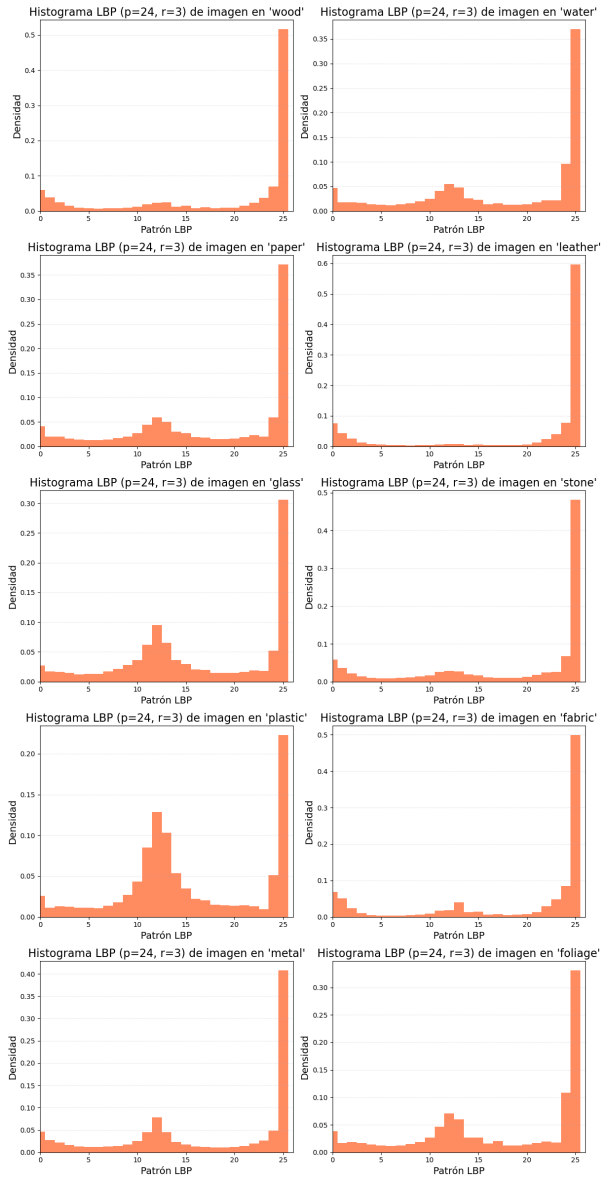


Fig. 12. Histogramas LBP $(p = 24, r = 3)$

Por último, se utilizó el modelo SVM para clasificar las imágenes LBP y evaluar el rendimiento obtenido. Los resultados para ambas configuraciones ($[P = 16, R = 2]$ y $[P = 24, R = 3]$) resultaron muy similares, aunque muy por debajo de la exactitud del algoritmo LBP original para 8 vecinos. Esto puede verse a partir de las matrices de confusión de clases verdaderas vs predichas. Se observa que

En las siguientes matrices de confusión se observa un mayor nivel de intensidad fuera de las diagonales, lo que indica una peor clasificación entre clases. Idealmente, una clasificación precisa tendría valores altos únicamente en la diagonal, y valores bajos o nulos fuera de ella.

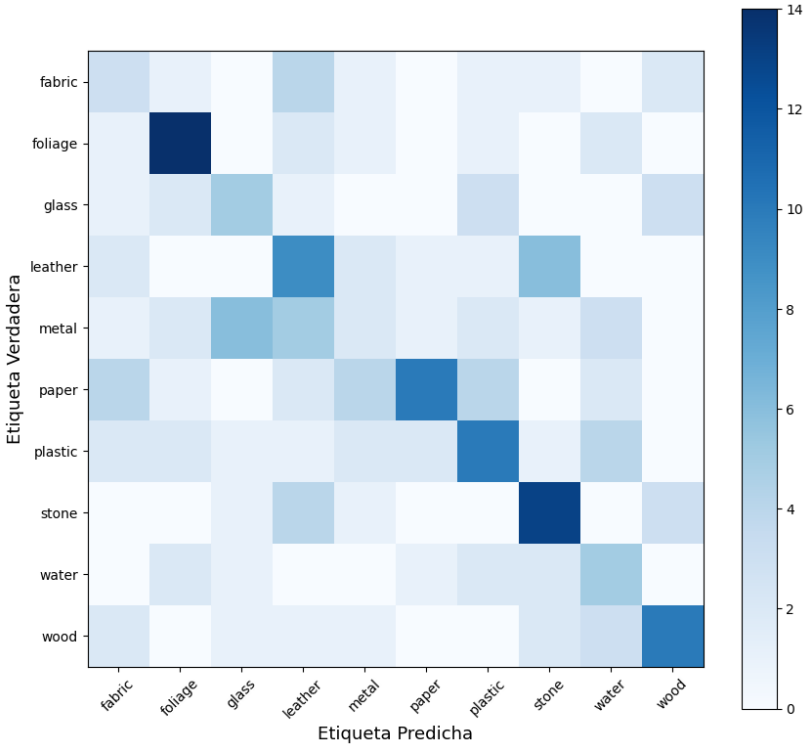


Fig. 13. Matriz de confusión con clasificación por SVM (p=2, r=16).

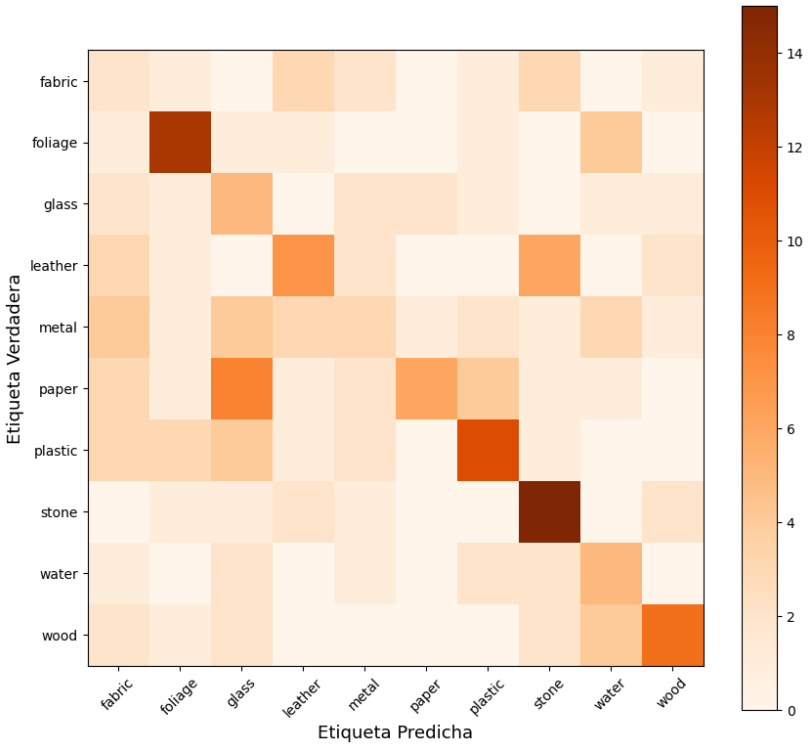


Fig. 14. Matriz de confusión con clasificación por SVM (p=3, r=24).

4 Código

El código para la realización de esta práctica se encuentra en el notebook RP_Práctica2.ipynb adjunto.

5 Conclusiones

En esta práctica se implementó el método de Patrones Binarios Locales (LBP) para la extracción de características. A partir del análisis de los histogramas LBP generados, se compararon distintos modelos de clasificación y se evaluó su exactitud. Basado en este proceso, se obtuvieron las siguientes conclusiones:

- Debido a que las imágenes dentro de una misma clase presentaban grandes diferencias de iluminación y contraste, el método de Patrones Binarios Locales obtuvo resultados mayormente deficientes, pues este es muy sensible a cambios de intensidad entre pixeles. Particularmente, clases con patrones visuales más consistentes como 'foliage' o 'stone' facilitaron una clasificación más precisa en comparación a clases como 'metal' o 'fabric'.
- El clasificador de Máquinas de Soporte Vectorial (SVM) superó en rendimiento al clasificador por vecinos más cercanos (KNN). Esto se atribuye a que, ante una mayor dimensionalidad de características, KNN pierde eficacia para distinguir cercanía. Este comportamiento se puede notar en clases como 'water', que registró aumento de 22 puntos porcentuales de precisión y de 17 en el f-score.
- En problemas de clasificación de múltiples clases, la similitud entre categorías puede comprometer la clasificación general, como ocurrió entre las clases 'leather' y 'foliage', cuyos patrones resultaron tan similares que el modelo tuvo dificultades para distinguirlos entre sí.
- El incremento de número de vecinos P y radio R en el algoritmo LBP no necesariamente se traduce en una mejor exactitud, pues para esta práctica, se obtuvieron resultados inferiores con las configuraciones $P = 16, R = 2$ y $P = 24, R = 3$ a comparación del algoritmo original de 8 vecinos.

6 Referencias

[1] Gonzalez, R., & Woods, R. (2008). *Digital Image Processing*. Prentice Hall.

[2] Pratt, W. K. (2001). *Digital Image Processing*. John Wiley and Sons Inc.

[3] Jahne, B. (2005). *Digital Image Processing*. Springer.

[4] Sánchez Garreta, J. S., & García, V. (2022). Special Issue on Data Preprocessing in Pattern Recognition: Recent Progress, Trends and Applications.